

\$sort (aggregation stage)

Definition

`$sort` Sorts all input documents and returns them to the pipeline in sorted order.

Compatibility

`$sort` You can use `$sort` for deployments hosted in the following environments:

- MongoDB Atlas: The fully managed service for MongoDB deployments in the cloud
- MongoDB Enterprise: The subscription-based, self-managed version of MongoDB
- MongoDB Community: The source-available, free-to-use, and self-managed version of MongoDB

Syntax

The `$sort` stage has the following prototype form:

```
{ $sort: { <field1>: <sort order>, <field2>: <sort order> ... } }
```

`$sort` takes a document that specifies the field(s) to sort by and the respective sort order.

`<sort order>` can have one of the following values:

Value	Description
<code>`1`</code>	Sort ascending.
<code>`-1`</code>	Sort descending.
<code>`{ \$meta: "textScore" }`</code>	order. See Text Score Metadata Sort for an example.

If sorting on multiple fields, sort order is evaluated from left to right. For example, in the form above, documents are first sorted by `<field1>`. Then documents with the same `<field1>` values are further sorted by `<field2>`.

Behavior

Performance

`$sort` `$sort` is a blocking stage, which causes the pipeline to wait for all input data to be retrieved for the blocking stage before processing the data. A blocking stage may reduce performance because it reduces parallel processing for a pipeline with multiple stages. A blocking stage may also use substantial amounts of memory for large data sets.

Limits

- You can sort on a maximum of 32 keys.
- Providing a sort pattern with duplicate fields causes an error.

Sort Consistency

MongoDB does not store documents in a collection in a particular order. When sorting on a field which contains duplicate values, documents containing those values may be returned in any order.

The `$sort` operation is not a "stable sort," which means that documents with equivalent sort keys are not guaranteed to remain in the same relative order in the output as they were in the input.

If the field specified in the sort criteria does not exist in two documents, then the value on which they are sorted is the same. The two documents may be returned in any order.

If consistent sort order is desired, include at least one field in your sort that contains unique values. The easiest way to guarantee this is to include the `_id` field in your sort query.

Consider the following `restaurant` collection:

```
db.restaurants.insertMany( [  
  { _id: 1, name: "Central Park Cafe", borough: "Manhattan"},  
  { _id: 2, name: "Rock A Feller Bar and Grill", borough: "Queens"},  
  { _id: 3, name: "Empire State Pub", borough: "Brooklyn"},  
  { _id: 4, name: "Stan's Pizzeria", borough: "Manhattan"},  
  { _id: 5, name: "Jane's Deli", borough: "Brooklyn"},  
] )
```

The following command uses the `$sort` stage to sort on the `borough` field:

```
db.restaurants.aggregate(  
  [  
    { $sort : { borough : 1 } }  
  ]  
)
```

In this example, sort order may be inconsistent, since the `borough` field contains duplicate values for both `Manhattan` and `Brooklyn`. Documents are returned in alphabetical order by `borough`, but the order of those documents with duplicate values for `borough` might not be the same across multiple executions of the same sort. For example, here are the results from two different executions of the above command:

```
{ _id: 3, name: "Empire State Pub", borough: "Brooklyn" }  
{ _id: 5, name: "Jane's Deli", borough: "Brooklyn" }  
{ _id: 1, name: "Central Park Cafe", borough: "Manhattan" }  
{ _id: 4, name: "Stan's Pizzeria", borough: "Manhattan" }  
{ _id: 2, name: "Rock A Feller Bar and Grill", borough: "Queens" }  
  
{ _id: 5, name: "Jane's Deli", borough: "Brooklyn" }  
{ _id: 3, name: "Empire State Pub", borough: "Brooklyn" }  
{ _id: 4, name: "Stan's Pizzeria", borough: "Manhattan" }  
{ _id: 1, name: "Central Park Cafe", borough: "Manhattan" }  
{ _id: 2, name: "Rock A Feller Bar and Grill", borough: "Queens" }
```

While the values for `borough` are still sorted in alphabetical order, the order of the documents containing duplicate values for `borough` (i.e. `Manhattan` and `Brooklyn`) is not the same.

To achieve a *consistent sort*, add a field which contains exclusively unique values to the sort. The following command uses the `$sort` stage to sort on both the `borough` field and the `_id` field:

```
db.restaurants.aggregate(  
  [  
    { $sort : { borough : 1, _id: 1 } }  
  ]  
)
```

Since the `_id` field is always guaranteed to contain exclusively unique values, the returned sort order will always be the same across multiple executions of the same sort.

Sort by an Array Field

When MongoDB sorts documents by an array-value field, the sort key depends on whether the sort is ascending or descending:

- In an ascending sort, the sort key is the lowest value in the array.
- In a descending sort, the sort key is the highest value in the array.

The query filter does not affect sort key selection.

For example, create a `shoes` collection with these documents:

```
db.shoes.insertMany( [
  { _id: 'A', sizes: [ 7, 11 ] },
  { _id: 'B', sizes: [ 8, 9, 10 ] }
] )
```

The following queries sort the documents by the `sizes` field in ascending and descending order:

```
// Ascending sort
db.shoes.aggregate( [
  {
    $sort: { sizes: 1 }
  }
] )

// Descending sort
db.shoes.aggregate( [
  {
    $sort: { sizes: -1 }
  }
] )
```

Both of the preceding queries return the document with `_id: 'A'` first because sizes `7` and `11` are the lowest and highest in the entries in the `sizes` array, respectively.

Filter and Sort by an Array Field

When you filter and sort by a field that contains an array, the filter does not affect the value used as the sort key. The sort always considers all array values as potential sort keys.

For example, the following query finds shoes with sizes greater than 9 and sorts the results by size in ascending order:

```
db.shoes.aggregate( [
  {
    $match: { sizes: { $gt: 9 } }
  },
  {
    $sort: { sizes: 1 }
  }
] )
```

The sort is ascending, which means that the sort key is the lowest value in the `sizes` array:

- In document `_id: 'A'`, the lowest `sizes` element is `7`. This value is used as the sort key even though it does not match the filter `{ sizes: { $gt: 9 } }`.
- In document `_id: 'B'`, the lowest `sizes` element is `8`. Similarly, this value is used as the sort key even though it does not match the filter.

The query returns the document with `_id: 'A'` first.

To only consider matched values as potential sort keys, you can generate a new field containing the matched values and sort on that field. For more information, see these pipeline stages and expressions:

- `$addField`
- `$filter`

`$sort` Operator and Memory

`$sort` + `$limit` Memory Optimization

When a `$sort` precedes a `$limit` and there are no intervening stages that modify the number of documents, the optimizer can coalesce the `$limit` into the `$sort`. This allows the `$sort` operation to only maintain the top `n` results as it progresses, where `n` is the specified limit, and ensures that MongoDB only needs to store `n` items in memory. This optimization still applies when `allowDiskUse` is `true` and the `n` items exceed the aggregation memory limit.

Optimizations are subject to change between releases.

`$sort` and Memory Restrictions

Starting in MongoDB 6.0, pipeline stages that require more than 100 megabytes of memory to execute write temporary files to disk by default. These temporary files last for the duration of the pipeline execution and can influence storage space on your instance. In earlier versions of MongoDB, you must pass `{ allowDiskUse: true }` to individual `find` and `aggregate` commands to enable this behavior.

Individual `find` and `aggregate` commands can override the `allowDiskUseByDefault` parameter by either:

- Using `{ allowDiskUse: true }` to allow writing temporary files out to disk when `allowDiskUseByDefault` is set to `false`
- Using `{ allowDiskUse: false }` to prohibit writing temporary files out to disk when `allowDiskUseByDefault` is set to `true`

For MongoDB Atlas, it is recommended to configure storage auto-scaling to prevent long-running queries from filling up storage with temporary files.

If your Atlas cluster uses storage auto-scaling, the temporary files may cause your cluster to scale to the next storage tier.

For additional details, see [Aggregation Pipeline Limits](#).

`$sort` Operator and Performance

The `$sort` operator can take advantage of an index if it's used in the first stage of a pipeline or if it's only preceded by a `$match` stage.

When you use the `$sort` on a sharded cluster, each shard sorts its result documents using an index where available. Then the `mongos` or one of the shards performs a streamed merge sort.

Examples

Ascending/Descending Sort

For the field or fields to sort by, set the sort order to `1` or `-1` to specify an ascending or descending sort respectively, as in the following example:

```
db.users.aggregate(  
  [  
    { $sort : { age : -1, posts: 1 } }  
  ]  
)
```

This operation sorts the documents in the `users` collection, in descending order according by the `age` field and then in ascending order according to the value in the `posts` field.

When comparing values of different BSON types in sort operations, MongoDB uses the following comparison order, from lowest to highest:

1. MinKey (internal type)
2. Null
3. Numbers (ints, longs, doubles, decimals)
4. Symbol, String
5. Object
6. Array
7. BinData
8. ObjectId
9. Boolean
10. Date
11. Timestamp
12. Regular Expression
13. JavaScript Code
14. JavaScript Code with Scope
15. MaxKey (internal type)

For details on the comparison/sort order for specific types, see [Comparison/Sort Order](#).

Text Score Metadata Sort

`$text` provides text query capabilities for self-managed (non-Atlas) deployments. For data hosted on MongoDB, MongoDB also offers an improved full-text query solution, MongoDB Search.

For a pipeline that includes `$text`, you can sort by descending relevance score using the `{ $meta: "textScore" }` expression. In the `{ <sort-key> }` document, set the `{ $meta: "textScore" }` expression to an arbitrary field name. The field name is ignored by the query system. For example:

```
db.users.aggregate([
  { $match: { $text: { $search: "operating" } } },
  { $sort: { score: { $meta: "textScore" }, posts: -1 } }
])
```

This operation uses the `$text` operator to match the documents, and then sorts first by the `"textScore"` metadata in descending order, and then by the `posts` field in descending order. The `score` field name in the sort document is ignored by the query system. In this pipeline, the `"textScore"` metadata is not included in the projection and is not returned as part of the matching documents. See `$meta` for more information.

The C# examples on this page use the `sample_mflix` database from the Atlas sample datasets. To learn how to create a free MongoDB Atlas cluster and load the sample datasets, see [Get Started in the MongoDB .NET/C# Driver documentation](#).

The following `Movie` class models the documents in the `sample_mflix.movies` collection:


```

public class Movie
{
    public ObjectId Id { get; set; }

    public int Runtime { get; set; }

    public string Title { get; set; }

    public string Rated { get; set; }

    public List<string> Genres { get; set; }

    public string Plot { get; set; }

    public ImdbData Imdb { get; set; }

    public int Year { get; set; }

    public int Index { get; set; }

    public string[] Comments { get; set; }

    [BsonElement("lastupdated")]
    public DateTime LastUpdated { get; set; }
}

```

The C# classes on this page use Pascal case for their property names, but the field names in the MongoDB collection use camel case. To account for this difference, you can use the following code to register a `ConventionPack` when your application starts:

```

var camelCaseConvention = new ConventionPack { new CamelCaseElementNameConvention() };
ConventionRegistry.Register("CamelCase", camelCaseConvention, type => true);

```

```
$sort
```

```
Sort()
```

sorts the input `Movie` documents first in descending order by the `Year` field, then in ascending order by the `Title` field:

To use the MongoDB .NET/C# driver to add a `$sort` stage to an aggregation pipeline, call the `Sort()` method on a `PipelineDefinition` object.

The following example creates a pipeline stage that sorts the input `Movie` documents first in descending order by the `Year` field, then in ascending order by the `Title` field:

```
var pipeline = new EmptyPipelineDefinition<Movie>()  
    .Sort(Builders<Movie>.Sort.Combine(  
        Builders<Movie>.Sort.Descending(m => m.Year),  
        Builders<Movie>.Sort.Ascending(m => m.Title)));
```

The Node.js examples on this page use the `sample_mflix` database from the Atlas sample datasets. To learn how to create a free MongoDB Atlas cluster and load the sample datasets, see [Get Started in the MongoDB Node.js driver documentation](#).

`$sort`

sorts the input `movie` documents first in descending order by the `year` field, and then in ascending order by the `title` field

To use the MongoDB Node.js driver to add a `$sort` stage to an aggregation pipeline, use the `$sort` operator in a pipeline object.

The following example creates a pipeline stage that sorts the input `movie` documents first in descending order by the `year` field, and then in ascending order by the `title` field. The example then runs the aggregation pipeline:

```
const pipeline = [{ $sort: { year: -1, title: 1 } }];  
  
const cursor = collection.aggregate(pipeline);  
return cursor;
```

Learn More

To see full aggregation examples that use the `$sort` stage, see the [Complete Aggregation Pipeline Tutorials](#).